ICPC Code Template

喵喵喵幼儿园

10 CATEGORIES **34** TEMPLATES **1767** SOURCE LINES

Quick Index

1 Data Structures **2**
1.1 01 树状数组 2
1.2 O(1) 查询树状数组 2
1.3 LCT 3
1.4 Top Tree 4
2 Geometry **5**
2.1 Geometry basic 5
3 Graph / Bipartite **8**
3.1 匈牙利算法 8
4 Graph / Flow **8**
4.1 最大流 8
4.2 最大流 9
4.3 最大流 10
4.4 最小费用最大流 10
4.5 最小费用最大流 12
5 Graph / Tarjan **12**
5.1 桥 12
5.2 割点 13
5.3 强连通分量 13
5.4 2-SAT 13
6 Math **14**
6.1 Binary GCD 14
6.2 det 14
7 Math / Number Theory **14**
7.1 杜教筛 14
7.2 Euclid 15
7.3 exgcd 16
7.4 min25 16
8 Misc **17**
8.1 Default(Yzj) 17
8.2 快速模乘 17
9 Poly **17**
9.1 多项式求逆 17
9.2 多项式乘法 19
9.3 计数常用代码 20
9.4 DIF-DIT NTT & All op 22
10 String **24**
10.1 广义 SAM 24
10.2 manacher 25
10.3 PAM 25
10.4 SA 26
10.5 后缀数组 27
10.6 SAM 27
10.7 后缀树 28

1 Data Structures

1.1 01 树状数组

FILE data-structures/fwt_bitset.cpp **LINES** 41

USE 下标从 1 开始，激活位置并查询区间内已激活位置数 **COST** Add/query O(log(N/64)), in-block query O(1)

AUTHOR ppip

```
struct fwt {
    static constexpr int B{(N>>6)+5};
    int t[B],n;
    unsigned long long b[B];
    void clear() {
        n=(::n>>6)+1;
        fill_n(t,n+1,0);
        fill_n(b,n+1,0);
    }
    int px(int x) {
        int y{x&63},z{x>>6};
        return __builtin_popcountll(b[z]&(~0ull>>(63-y)));
    }
    int qry(int l,int r) {
        --l;
        int ans{px(r)-px(l)};
        l>>=6;r>>=6;
        while (r>l) ans+=t[r],r&=r-1;
        while (l>r) ans-=t[l],l&=l-1;
        return ans;
    }
    void add(int x) {
        unsigned long long w{1ull<<(x&63)};
        if (b[x>>6]&w) return;
        b[x>>6]|=w;
        x=(x>>6)+1;
        while (x<=n) {
            ++t[x];
            x+=x&-x;
        }
    }
    int qry(int x) {
        int ans{px(x)};
        x>>=6;
        while (x) {
            ans+=t[x];
            x&=x-1;
        }
        return ans;
    }
} t;
```

1.2 O(1) 查询树状数组

FILE data-structures/fwt_suffix.cpp **LINES** 16

USE 下标从 1 开始，单点加、查询后缀和；适合查询远多于修改 **COST** Query O(1), add O(128+N/4096)

AUTHOR ppip

```
struct FWT {
    int s1[N+5],s2[(N>>6)+5],s3[(N>>12)+5];
    void init() {
        memset(s1,0,sizeof(s1));
        memset(s2,0,sizeof(s2));
        memset(s3,0,sizeof(s3));
    }
    void add(int x,int y) {
        for (int i{(x>>6)<<6};i<=x;++i) s1[i]+=y;
        for (int i{(x>>12)<<6};i<(x>>6);++i) s2[i]+=y;
        for (int i{0};i<(x>>12);++i) s3[i]+=y;
    }
    int qry(int x) {
        return s1[x]+s2[x>>6]+s3[x>>12];
    }
} T;
```

1.3 LCT

FILE data-structures/lct.cpp **LINES** 102

USE LCT **COST** O(nlogn)

AUTHOR ysy

```cpp
#include<iostream>
#include<cstdio>
using namespace std;
int read(){
    int x=0,f=1;char ch=getchar();
    while(ch<'0'||ch>'9'){if(ch=='-')f=-1;ch=getchar();}
    while(ch>='0'&&ch<='9'){x=x*10+ch-'0';ch=getchar();}
    return x*f;
}
const int N=100010;
int n,q,fa[N],ch[N][2],rv[N],v[N],yh[N];
bool nrt(int x){return(ch[fa[x]][0]==x||ch[fa[x]][1]==x);}
void pup(int x){
    yh[x]=(yh[ch[x][0]]^yh[ch[x][1]]^v[x]);
}
void rnt(int x){
    int y=fa[x],z=fa[y],k=(ch[y][1]==x),w=ch[x][k^1];
    if(nrt(y))ch[z][ch[z][1]==y]=x;
    fa[fa[fa[ch[ch[x][k^1]=y][k]=w]=y]=x]=z;
    pup(y);pup(x);
}
void pd(int x){
    swap(ch[x][0],ch[x][1]);
    rv[x]^=1;
}
void psd(int x){
    if(rv[x]){
        pd(ch[x][0]);
        pd(ch[x][1]);
        rv[x]=0;
    }
}
int s[N],tt;
void psc(int x){
    s[++tt]=x;
    for(;nrt(x);x=fa[x])s[++tt]=fa[x];
    while(tt)psd(s[tt--]);
}
void spl(int x){
    psc(x);
    while(nrt(x)){
        int y=fa[x],z=fa[y];
        if(nrt(y))rnt((ch[z][1]==y)^(ch[y][1]==x)?x:y);
        rnt(x);
    }
}
void acs(int x){
    int y=0;
    while(x){
        spl(x);
        ch[x][1]=y;
        pup(x);
        y=x;x=fa[x];
    }
}
void mrt(int x){
    acs(x);
    spl(x);
    pd(x);
}
int grt(int x){
    acs(x);
    spl(x);
    while(ch[x][0])x=ch[x][0];
    spl(x);
    return x;
}
bool chk(int x,int y){
    mrt(x);
    return(grt(y)==x);
```

```
}
void lnk(int x,int y){
    if(chk(x,y))return;
    fa[x]=y;
}
void cut(int x,int y){
    if(!chk(x,y)||ch[x][1]!=y||ch[y][0])return;
    ch[x][1]=fa[y]=0;pup(x);
}
void qry(int x,int y){
    if(!chk(x,y))cout<<"?\n";
    cout<<yh[x]<<"\n";
}
void upd(int x,int y){
    spl(x);
    v[x]=y;
    pup(x);
}
int main(){
    n=read(),q=read();
    for(int i=1;i<=n;i++){
        v[i]=read();
    }
    while(q--){
        int op=read(),x=read(),y=read();
        if(op==0)qry(x,y);
        if(op==1)lnk(x,y);
        if(op==2)cut(x,y);
        if(op==3)upd(x,y);
    }
    return 0;
}
```

1.4 Top Tree

FILE data-structures/toptree.cpp **LINES** 60

USE Magic tree that can fulfill every wish. **COST** Tree height O(log n)

AUTHOR ppip

```
vector<pair<int,int>> e[N+5];
int sz[N+5],fa[N+5],son[N+5];
void TOPTREE_INIT(int u) {
    sz[u]=1;
    for (auto [v,_]:e[u])
        if (v!=fa[u]) {
            TOPTREE_INIT(v);
            sz[u]+=sz[v];
            if (sz[v]>sz[son[u]]) son[u]=v;
        }
    if (son[u]) {
        int si{exchange(sz[son[u]],0)};
        sort(e[u].begin(),e[u].end(),[](auto x,auto y){return sz[x.first]<sz[y.first];});
        sz[son[u]]=si;
    }
}
struct cluster {
    //....
} t[N*2+5];
int cnode;
using P=pair<int,int>;
using iter=vector<P>::iterator;
int merge(iter L,iter R,char tp) {
    if (L+1==R) {
        int z{L->first};
        if (z<=n) t[z].u=fa[z],t[z].v=z;
        return z;
    }
    iter p{L+1};
    int sum{accumulate(p,R,-L->second,[](int x,P y){return x+y.second;})};
    while (p+1!=R&&sum>p->second) {
        sum-=p->second*2;
        ++p;
    }
    int u{++cnode};
    t[u].t=tp;
    t[u].l=merge(L,p,tp);
```

```
    t[u].r=merge(p,R,tp);
    t[t[u].l].f=t[t[u].r].f=u;
    t[u].u=t[t[u].l].u;
    t[u].v=(tp=='C'?t[t[u].r].v:t[t[u].l].v);
    return u;
}
int build(int u) {
    vector<P> v1;v1.emplace_back(u,1);
    while (son[u]) {
        vector<P> v2;v2.emplace_back(son[u],1);
        for (auto [v,_]:e[u])
            if (v!=fa[u]&&v!=son[u]) v2.emplace_back(build(v),sz[v]);
        v1.emplace_back(merge(v2.begin(),v2.end(),'R'),sz[u]-sz[son[u]]);
        u=son[u];
    }
    return merge(v1.begin(),v1.end(),'C');
}
int main() {
    cnode=n;
    TOPTREE_INIT(1);
    int rt{build(1)};
    return 0;
}
```

2 Geometry

2.1 Geometry basic

FILE Geometry/basic.cpp **LINES** 201

USE 分数、凸包、点在凸包内、闵可夫斯基和 **COST** O(n log n)

AUTHOR yzj

```
#include<bits/stdc++.h>

#define int long long
#define i128 __int128

#define vc vector
#define pb emplace_back
#define pii pair<int, int>
#define mkp make_pair
#define rep(i, a, b) for(int i = (a); i <= (b); ++i)
#define lep(i, a, b) for(int i = (a); i >= (b); --i)

using namespace std;

mt19937 gen(chrono::system_clock::now().time_since_epoch().count());

inline int read() {
        int x = 0, w = 0;
        char ch = getchar();
        while(!isdigit(ch)) w |= (ch == '-'), ch = getchar();
        while(isdigit(ch)) x = x * 10 + (ch ^ 48), ch = getchar();
        return w ? -x : x;
}

const int N = 2e5 + 5;

inline i128 mygcd (i128 x, i128 y) { return y ? mygcd (y, x % y) : x; }
inline i128 myabs (i128 x) { return x < 0 ? - x : x; }
struct Frac {
        i128 p, q;
        Frac (i128 pp = 0, i128 qq = 1) : p (pp), q (qq) {
                if (q < 0) p = - p, q = - q;
                assert (q != 0);
                i128 g = mygcd (myabs (p), q);
                p /= g; q /= g;
        }

        Frac operator + (const Frac &a) const {
                i128 g = mygcd(q, a.q);
                return Frac(p * (a.q / g) + a.p * (q / g),
                        (q / g) * a.q);
        }

```

```
        Frac operator - (const Frac &a) const {
                i128 g = mygcd(q, a.q);
                return Frac(p * (a.q / g) - a.p * (q / g),
                        (q / g) * a.q);
        }

        Frac operator * (const Frac &a) const {
                i128 g1 = mygcd(myabs(p), a.q);
                i128 g2 = mygcd(myabs(a.p), q);
                return Frac((p / g1) * (a.p / g2),
                        (q / g2) * (a.q / g1));
        }

        Frac operator / (const Frac &a) const {
                assert(a.p != 0);
                i128 g1 = mygcd(myabs(p), myabs(a.p));
                i128 g2 = mygcd(q, a.q);
                return Frac((p / g1) * (a.q / g2),
                        (q / g2) * (a.p / g1));
        }

        bool operator < (const Frac &a) const {
                return (i128) p * a.q < (i128) a.p * q;
        }
        bool operator == (const Frac &a) const {
                return p == a.p && q == a.q;
        }
} ;

namespace Geometry {
        const double eps = 1e-7, PI = acos (- 1);
        struct Point {
                int x, y;
                Point (int x = 0, int y = 0): x (x), y (y) {}
                auto operator<=>(const Point&) const = default;
                Point operator - (const Point &a) const { return Point (x - a.x, y - a.y); }
                Point operator + (const Point &a) const { return Point (x + a.x, y + a.y); }
        } ;
        inline int half (Point a) { return a.y > 0 || (a.y == 0 && a.x > 0); }
        inline int crs (Point a, Point b) { return a.x * b.y - a.y * b.x; }
        inline int dot (Point a, Point b) { return a.x * b.x + a.y * b.y; }
        inline int cmp (Point a, Point b) {
                return half (a) == half (b) ? crs (a, b) > 0 : half (a) < half (b);
        }
        inline vc<Point> ConvexHull (vc <Point> a) {
                sort (a.begin (), a.end ());
                a.erase (unique (a.begin (), a.end ()), a.end ());
                vc <Point> b;
                if (a.size () <= 1) return a;
                rep (t, 0, 1) {
                        int k = b.size ();
                        for (auto u : a) {
                                while ((b.size () >= k + 2) && crs (b.back () - b[b.size () - 2], u - b.back
()) <= 0) b.pop_back ();
                                b.pb (u);
                        }
                        b.pop_back ();
                        reverse (a.begin (), a.end ());
                }
                return b;
        }
        inline int inarea (const vc <Point> &a, Point u) {
                if (a.size () < 2) return 0;
                int pos = 0, neg = 0;
                for (int i = 0; i < a.size (); i ++) {
                        int v = crs (a[(i + 1) % a.size ()] - a[i], u - a[i]);
                        pos += (v > 0); neg += (v < 0);
                }
                return (pos == a.size ()) || (neg == a.size ());
        }

        inline bool onseg(Point a, Point b, Point p) {
                if (crs(b - a, p - a) != 0) return false;
                return min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x)
                && min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
```

```
118         }
119
120         // 要求 a 是逆时针凸包，不重复首尾顶点
121         // 0: outside, 1: boundary, 2: inside
122         inline int inConvex(const vc<Point> &a, Point p) {
123                 int n = a.size();
124                 if (n == 0) return 0;
125                 if (n == 1) return p == a[0] ? 1 : 0;
126                 if (n == 2) return onseg(a[0], a[1], p) ? 1 : 0;
127
128                 int x = crs(a[1] - a[0], p - a[0]);
129                 int y = crs(a[n - 1] - a[0], p - a[0]);
130
131                 if (x < 0 || y > 0) return 0;
132                 if (x == 0) return onseg(a[0], a[1], p) ? 1 : 0;
133                 if (y == 0) return onseg(a[0], a[n - 1], p) ? 1 : 0;
134
135                 int l = 1, r = n - 1;
136                 while (l + 1 < r) {
137                         int mid = (l + r) >> 1;
138                         if (crs(a[mid] - a[0], p - a[0]) >= 0)
139                                 l = mid;
140                         else r = mid;
141                 }
142
143                 int s = crs(a[l + 1] - a[l], p - a[l]);
144                 if (s < 0) return 0;
145                 if (s == 0) return onseg(a[l], a[l + 1], p) ? 1 : 0;
146                 return 2;
147         }
148         inline vc<Point> Minkowski(vc<Point> a, vc<Point> b) {
149                 if (a.empty() || b.empty()) return {};
150                 if (a.size() == 1) {
151                         for (auto &p : b) p = p + a[0];
152                         return b;
153                 }
154                 if (b.size() == 1) {
155                         for (auto &p : a) p = p + b[0];
156                         return a;
157                 }
158
159                 auto cmp = [](Point a, Point b) {
160                         return a.y != b.y ? a.y < b.y : a.x < b.x;
161                 };
162                 auto angleCmp = [](Point a, Point b) {
163                         if (half(a) != half(b))
164                                 return half(a) > half(b) ? -1 : 1;
165                         i128 v = (i128)a.x * b.y - (i128)a.y * b.x;
166                         return v > 0 ? -1 : v < 0 ? 1 : 0;
167                 };
168
169                 rotate(a.begin(), min_element(a.begin(), a.end(), cmp), a.end());
170                 rotate(b.begin(), min_element(b.begin(), b.end(), cmp), b.end());
171
172                 int n = a.size(), m = b.size();
173                 int i = 0, j = 0;
174                 Point p = a[0] + b[0];
175
176                 vc<Point> c;
177                 while (i < n || j < m) {
178                         c.pb(p);
179
180                         if (i == n) {
181                                 p = p + b[(j + 1) % m] - b[j];
182                                 ++j;
183                         } else if (j == m) {
184                                 p = p + a[(i + 1) % n] - a[i];
185                                 ++i;
186                         } else {
187                                 Point u = a[(i + 1) % n] - a[i];
188                                 Point v = b[(j + 1) % m] - b[j];
189
190                                 int t = angleCmp(u, v);
191                                 if (t <= 0) p = p + u, ++i;
192                                 if (t >= 0) p = p + v, ++j;
```

```
                        }
                }
                return c;
        }
}

signed main () {
        return 0;
}
```

3 Graph / Bipartite

3.1 匈牙利算法

FILE graph/bipartite/hungary.cpp **LINES** 26

USE 二分图最大匹配 **COST** O(nm)

AUTHOR yzj

```
namespace bipartite {
        int vis[N], match[N], ans;

        inline bool hungary (int x) {
                for (auto y : G[x]) if (!vis[y]) {
                        vis[y] = 1;
                        if (! match[y] || hungary (match[y])) return match[y] = x, 1;
                }
                return 0;
        }

        inline void mian () {
                n = read (); int k = read (); m = read ();
                for (int i = 1, x, y; i <= m; i ++) {
                        x = read (); y = read ();
                        G[x].pb (y);
                }

                rep (i, 1, n) {
                        rep (j, 0, k) vis[j] = 0;
                        ans += hungary (i);
                }

                printf ("%lld\n", ans);
        }
}
```

4 Graph / Flow

4.1 最大流

FILE graph/flow/maxflow(ysy).cpp **LINES** 64

USE 网络最大流 **COST** O(nm^2), 二分图中 O (m \sqrt n)

AUTHOR ysy

```
#include<iostream>
#include<cstdio>
#include<queue>
#define int long long
using namespace std;
int read(){
    int x=0,f=1;char ch=getchar();
    while(ch<'0'||ch>'9'){if(ch=='-')f=-1;ch=getchar();}
    while(ch>='0'&&ch<='9'){x=x*10+ch-'0';ch=getchar();}
    return x*f;
}
const int N=220,M=10010,inf=1e18;
int n,m,s,t,fr[N],nex[M],to[M],v[M],ct=1,nw[N],dep[N],ans;
queue<int>q;
void add(int x,int y,int z){
    nex[++ct]=fr[x];
    to[ct]=y;
    v[ct]=z;
    fr[x]=ct;
}
void lnk(int x,int y,int z){
    add(x,y,z);add(y,x,0);
```

```
}
bool bfs(){
    for(int i=1;i<=n;i++){
        dep[i]=0,nw[i]=fr[i];
    }
    q.push(s);dep[s]=1;
    while(!q.empty()){
        int x=q.front();q.pop();
        for(int i=fr[x];i;i=nex[i]){
            int y=to[i];
            if(v[i]&&!dep[y]){
                dep[y]=dep[x]+1;
                q.push(y);
            }
        }
    }
    return dep[t];
}
int dfs(int x,int fl){
    if(x==t)return fl;
    int res=0;
    for(int i=nw[x];i&&fl;i=nex[i]){
        nw[x]=i;int y=to[i];
        if(v[i]&&dep[y]==dep[x]+1){
            int z=dfs(y,min(v[i],fl));
            fl-=z;res+=z;
            v[i]-=z;v[i^1]+=z;
        }
    }
    if(!res)dep[x]=0;
    return res;
}
signed main(){
    n=read(),m=read(),s=read(),t=read();
    for(int i=1;i<=m;i++){
        int u=read(),v=read(),w=read();
        lnk(u,v,w);
    }
    while(bfs())ans+=dfs(s,inf);
    cout<<ans;
    return 0;
}
```

4.2 最大流

FILE graph/flow/maxflow(yzj).cpp **LINES** 27

USE 网络最大流 **COST** O(nm^2), 二分图中 O (m \sqrt n)

AUTHOR yzj

```
namespace flow {
    int dep[N], S, T;
    ll maxflow;
    int hd[N], cur[N], to[N << 1], ne[N << 1], e[N << 1], tc = 1;
    inline void add(int x, int y, int z) {
        to[++tc] = y; ne[tc] = hd[x]; hd[x] = tc; e[tc] = z;
        to[++tc] = x; ne[tc] = hd[y]; hd[y] = tc; e[tc] = 0;
    }
    inline int bfs() {
        rep(i, 0, T) dep[i] = -1; dep[S] = 0; queue<int> Q; Q.push(S);
        while(!Q.empty()) {
            int x = Q.front(); Q.pop(); cur[x] = hd[x];
            if(x == T) return 1;
            for(int i = hd[x], y; i; i = ne[i])
                if(e[i] && dep[y = to[i]] == -1) dep[y] = dep[x] + 1, Q.push(y);
        }
        return 0;
    }
    inline int dfs(int x, int fl) {
        if(x == T) return maxflow += fl, fl;
        for(int &i = cur[x], y, o; i; i = ne[i])
            if(e[i] && dep[y = to[i]] == dep[x] + 1 && (o = dfs(y, min(fl, e[i]))))
                return e[i] -= o, e[i ^ 1] += o, o;
        return 0;
    }
    inline void dinic() { while(bfs()) { while(dfs(S, inf)); } }
}
```

4.3 最大流

FILE graph/flow/maxflow.cpp **LINES** 60

USE Dinic, 要求汇点编号为最大值 **COST** 相信的心就是魔法

AUTHOR ppip

```cpp
struct edge {
    int u,v,w,nxt;
    edge(int _,int __,int ___,int ____):
        u{_},v{__},w{___},nxt{____} {}
    edge() {}
};
vector<edge> e(2);
int hd[N+5];
void _add(int u,int v,int w) {
    e.emplace_back(u,v,w,hd[u]);
    hd[u]=e.size()-1;
}
void add(int u,int v,int w) {
    _add(u,v,w);
    _add(v,u,0);
}
int q[N+5];
int dep[N+5],cur[N+5];
bool bfs(int s,int t) {
    dep[s]=1;
    int l{0},r{0};
    q[r++]=s;
    while (r-l) {
        int u{q[l++]};
        for (int x{hd[u]};x;x=e[x].nxt)
            if (!dep[e[x].v]&&e[x].w) {
                dep[e[x].v]=dep[u]+1;
                q[r++]=e[x].v;
            }
    }
    return dep[t];
}
long long dfs(int s,int t,long long flow) {
    if (s==t||!flow) return flow;
    long long ans{0};
    for (int &x{cur[s]},d;x;x=e[x].nxt) {
        if (dep[e[x].v]==dep[s]+1&&e[x].w&&(d=dfs(e[x].v,t,min(flow,(long long)e[x].w)))) {
            flow-=d;
            ans+=d;
            e[x].w-=d;
            e[x^1].w+=d;
            if (!flow) return ans;
        }
    }
    return ans;
}
long long calc(int s,int t) {
    fill_n(dep+1,t,0);
    long long flow{0};
    while (bfs(s,t)) {
        copy(hd+1,hd+t+1,cur+1);
        flow+=dfs(s,t,LLONG_MAX);
        fill_n(dep+1,t,0);
    }
    return flow;
}
void clear(int t) {
    e.resize(2);
    fill_n(hd+1,t,0);
}
```

4.4 最小费用最大流

FILE graph/flow/mcmf(ysy).cpp **LINES** 95

USE 最小费用最大流，原始对偶实现 **COST** /yun

AUTHOR ysy

```cpp
#include<iostream>
#include<cstdio>
#include<queue>
#define M(x,y) make_pair(x,y)
```

```cpp
using namespace std;
int read(){
    int x=0,f=1;char ch=getchar();
    while(ch<'0'||ch>'9'){if(ch=='-')f=-1;ch=getchar();}
    while(ch>='0'&&ch<='9'){x=x*10+ch-'0';ch=getchar();}
    return x*f;
}
const int N=100010,inf=1e9+7;
int n,m,s,t,fr[N],nex[N],to[N],fl[N],vl[N],ct=1;
int h[N],d[N],b[N],bl[N],eg[N],cnt,res;
queue<int>Q;
priority_queue<pair<int,int> >q;
void add(int x,int y,int z,int f){
    nex[++ct]=fr[x];
    to[ct]=y;
    vl[ct]=z;
    fl[ct]=f;
    fr[x]=ct;
}
void lnk(int x,int y,int z,int f){
    add(x,y,z,f);add(y,x,-z,0);
}
void spfa(){
    for(int i=1;i<=n;i++){
        h[i]=inf;
    }
    h[s]=0;Q.push(s);b[s]=1;
    while(!Q.empty()){
        int x=Q.front();Q.pop();
        b[x]=0;
        for(int i=fr[x];i;i=nex[i]){
            if(!fl[i])continue;
            int y=to[i];
            if(h[y]>h[x]+vl[i]){
                h[y]=h[x]+vl[i];
                if(!b[y]){
                    Q.push(y);
                    b[y]=1;
                }
            }
        }
    }
}
bool dij(){
    for(int i=1;i<=n;i++){
        d[i]=inf;b[i]=0;
    }
    d[s]=0;q.push(M(0,s));
    while(!q.empty()){
        int x=q.top().second;q.pop();
        if(b[x])continue;
        b[x]=1;
        for(int i=fr[x];i;i=nex[i]){
            if(!fl[i])continue;
            int y=to[i],z=vl[i]-(h[y]-h[x]);
            if(d[y]>d[x]+z){
                d[y]=d[x]+z;
                bl[y]=x,eg[y]=i;
                q.push(M(-d[y],y));
            }
        }
    }
    return(d[t]!=inf);
}
int main(){
    n=read(),m=read(),s=read(),t=read();
    for(int i=1;i<=m;i++){
        int u=read(),v=read(),w=read(),c=read();
        lnk(u,v,c,w);
    }
    spfa();
    while(dij()){
        for(int i=1;i<=n;i++){
            h[i]+=d[i];
        }
```

```
        int x=t,mn=inf,cs=0;
        while(x!=s){
            mn=min(mn,fl[eg[x]]);
            x=bl[x];
        }
        x=t;
        while(x!=s){
            cs+=vl[eg[x]]*mn;
            fl[eg[x]]-=mn;fl[eg[x]^1]+=mn;
            x=bl[x];
        }
        cnt+=mn,res+=cs;
    }
    cout<<cnt<<" "<<res<<"\n";
    return 0;
}
```

4.5 最小费用最大流

FILE graph/flow/mcmf.cpp **LINES** 38

COST O(nmF)

AUTHOR yzj

```
// @brief
namespace MCMF {
        const int MAXN = 1e6 + 5, inf = 1e18;
        int s, t, mxfl, mico;
        int inq[MAXN], d[MAXN], pr[MAXN];
        int hd[MAXN], cur[MAXN], to[MAXN << 1], ne[MAXN << 1], e[MAXN << 1], c[MAXN << 1], tc = 1;

        inline void add (int x, int y, int z, int w) {
                // cerr << x << " " << y << " " << z << " " << w << endl;
                to[++ tc] = y; ne[tc] = hd[x]; hd[x] = tc; e[tc] = z; c[tc] = w;
                to[++ tc] = x; ne[tc] = hd[y]; hd[y] = tc; e[tc] = 0; c[tc] = - w;
        }

        inline int spfa () {
                rep (i, 0, t) inq[i] = pr[i] = 0, d[i] = inf;
                queue <int> Q; Q.push (s); d[s] = 0; inq[s] = 1;
                while (!Q.empty ()) {
                        int x = Q.front (); Q.pop (); inq[x] = 0; cur[x] = hd[x];
                        for (int i = hd[x], y; i; i = ne[i])
                                if (e[i] && d[y = to[i]] > d[x] + c[i]) {
                                        d[y] = d[x] + c[i]; pr[y] = i;
                                        if (!inq[y]) Q.push (y), inq[y] = 1;
                                }
                }
                return d[t] != inf;
        }

        inline pii mcmf() {
        pii res = mkp(0, 0);
        while(spfa ()) {
            int c = inf;
            for(int x = t; x != s; x = to[pr[x] ^ 1]) c = min(c, e[pr[x]]);
            for(int x = t; x != s; x = to[pr[x] ^ 1]) e[pr[x]] -= c, e[pr[x] ^ 1] += c;
            res.first += c; res.second += d[t] * c;
        }
        return res;
    }
}
```

5 Graph / Tarjan

5.1 桥

FILE graph/tarjan/bridge.cpp **LINES** 13

USE Tarjan 求无向图中的桥

COST O(n+m)

AUTHOR yzj

```
namespace bridge {
        inline void tarjan (int x, int in) {
                dfn[x] = low[x] = ++ DDFN;
                for (int i = hd[x], y; i; i = ne[i]) if (i != in) {
                        if (! dfn[y = to[i]]) {
                                tarjan (y, i ^ 1);
```

```
                                low[x] = min (low[x], low[y]);
                                if (low[y] > dfn[x]) bg[i >> 1] = 1;
                        }
                        else low[x] = min (low[x], dfn[y]);
                }
        }
}
```

5.2 割点

FILE graph/tarjan/cut_vertex.cpp **LINES** 18

USE Tarjan 求无向图中的割点 **COST** O(n+m)

AUTHOR yzj

```
namespace cutnode {

        vc <int> ans;
        inline void tarjan (int x, int rt) {
                dfn[x] = low[x] = ++ DDFN; int cnt = 0, fl = 0;
                for (auto y : G[x])
                        if (! dfn[y]) {
                                tarjan (y, rt);
                                low[x] = min (low[x], low[y]);
                                if (low[y] >= dfn[x]) {
                                        ++ cnt;
                                        if (!fl && (x != rt || cnt > 1))
                                                fl = 1, ans.pb (x);
                                }
                        }
                        else low[x] = min (low[x], dfn[y]);
        }
}
```

5.3 强连通分量

FILE graph/tarjan/scc.cpp **LINES** 10

USE Tarjan 缩点，id 为所属强连通分量，w 为点权和 **COST** O(n+m)

AUTHOR yzj

```
inline void Tarjan(int x) {
    dfn[x]=low[x]=++idx; sta[++topf]=x; ins[x]=1;
    for(int i=hd[x],y;i;i=ne[i])
        if(!dfn[y=to[i]]) Tarjan(y), low[x]=min(low[x],low[y]);
        else if(ins[y]) low[x]=min(low[x],dfn[y]);
    if(dfn[x]==low[x]) {
        ++cnt; int z;
        do { z=sta[topf--]; id[z]=cnt; w[cnt]+=a[z]; ins[z]=0; }while(x^z);
    }
}
```

5.4 2-SAT

FILE graph/tarjan/two_sat.cpp **LINES** 22

USE Tarjan 强连通分量求解 2-SAT **COST** O(n+m)

AUTHOR yzj

```
namespace twosat {
        int tot;

        inline void tarjan (int x) {
                dfn[x] = low[x] = ++ DDFN; stk[++ topf] = x; ins[x] = 1;
                for (int i = hd[x], y; i; i = ne[i])
                        if (! dfn[y = to[i]]) tarjan (y), low[x] = min (low[x], low[y]);
                        else if (ins[y]) low[x] = min (low[x], dfn[y]);
                if (dfn[x] == low[x]) {
                        int z; ++ tot;
                        do { z = stk[topf --]; ins[z] = 0; id[z] = tot; } while (x != z);
                }
        }

        inline void getans () {
                /*
                rep (i, 1, n)
                        if (id[i true] < id[i false])
                                then i is true;
                */
        }
}
```

6 Math

6.1 Binary GCD

FILE math/binary_gcd.cpp **LINES** 6

USE 超快 gcd 算法 **COST** O(log n)，可以当作 O(1)

AUTHOR ppip

```cpp
#define ctz __builtin_ctz
inline int gcd(int a,int b) {
    int az{ctz(a)},bz{ctz(b)},z{min(az,bz)},t;b>>=bz;
    while (a) a>>=az,t=a-b,az=ctz(t),b=min(a,b),a=abs(t);
    return b<<z;
}
```

6.2 det

FILE math/det.cpp **LINES** 17

USE 行列式求值 **COST** O(n^3)

AUTHOR yzj

```cpp
inline mint det (vc <vc <mint> > a, int n) {
    int rev = 0;
    rep (i, 1, n) {
        rep (j, i + 1, n) {
            while (a[i][i].x) {
                int d = a[j][i].x / a[i][i].x;
                rep (k, i, n) a[j][k] -= a[i][k] * d;
                swap (a[i], a[j]); rev ^= 1;
            }
            swap (a[i], a[j]); rev ^= 1;
        }
        if (!a[i][i].x) return 0;
    }
    mint res = 1;
    rep (i, 1, n) res *= a[i][i];
    return rev ? MOD - res : res;
}
```

7 Math / Number Theory

7.1 杜教筛

FILE math/number_theory/dujiao.cpp **LINES** 65

USE 求欧拉函数与莫比乌斯函数的前缀和 **COST** O(n^{2/3})

AUTHOR yzj

```cpp
#include<iostream>
#include<cstdio>
#include<map>
#define ll long long
using namespace std;
int read(){
    int x=0,f=1;char ch=getchar();
    while(ch<'0'||ch>'9'){if(ch=='-')f=-1;ch=getchar();}
    while(ch>='0'&&ch<='9'){x=x*10+ch-'0';ch=getchar();}
    return x*f;
}
const int N=4000040,mod=20101009;
int t,mx=4000000;
int p[N],pri[N],cnt,mo[N],ans;
ll fi[N];
map<int,int>f1;
map<int,ll>f2;
void oula(){
    mo[1]=1,fi[1]=1;
    for(int i=2;i<=mx;i++){
        if(!fi[i]){
            pri[++cnt]=i;
            mo[i]=-1;
            fi[i]=i-1;
        }
        for(int j=1;j<=cnt&&i*pri[j]<=mx;j++){
            int p=pri[j];
            fi[i*p]=fi[i]*p;
            if(i%p==0)break;
```

```
            fi[i*p]=fi[i]*(p-1);
            mo[i*p]=-mo[i];
        }
    }
    for(int i=1;i<=mx;i++){
        mo[i]+=mo[i-1],fi[i]+=fi[i-1];
    }
}
int sumu(int n){
    if(n<=mx)return mo[n];
    if(f1[n])return f1[n];
    int cnt=1;
    for(ll l=2,r=0;l<=n;l=r+1){
        r=n/(n/l);
        cnt-=(r-l+1)*sumu(n/l);
    }
    return f1[n]=cnt;
}
ll sumf(int n){
    if(n<=mx)return fi[n];
    if(f2[n])return f2[n];
    ll cnt=n*(n+1ll)/2ll;
    for(ll l=2,r=0;l<=n;l=r+1){
        r=n/(n/l);
        cnt-=(r-l+1ll)*sumf(n/l);
    }
    return f2[n]=cnt;
}
signed main(){
    oula();
    t=read();
    while(t--){
        int n=read();
        cout<<sumf(n)<<" "<<sumu(n)<<"\n";
    }
}
```

7.2 Euclid

FILE math/number_theory/euclid.cpp **LINES** 44

USE 类欧几里得算法 **COST** O(\log \max(a,c))

AUTHOR yzj

```
namespace Euclid {
        using ll = long long;
        const mint inv2 = 499122177, inv6 = 166374059;
        struct Data { mint f, g, h; };

        inline Data calc (ll n, ll a, ll b, ll c) {
                if(n < 0) return {};

                ll A = a / c, B = b / c;
                a %= c; b %= c;
                if(a < 0) a += c, --A;
                if(b < 0) b += c, --B;

                mint N = n % MOD, s0 = N + 1;
                mint s1 = N * s0 * inv2;
                mint s2 = N * s0 * (2 * N + 1) * inv6;

                if(A || B) {
                        Data t = calc(n, a, b, c);
                        mint x = A % MOD, y = B % MOD;
                        return {
                                t.f + x * s1 + y * s0,
                                t.g + x * x * s2 + y * y * s0
                                        + 2 * x * t.h + 2 * y * t.f
                                        + 2 * x * y * s1,
                                t.h + x * s2 + y * s1
                        };
                }

                ll m = ((__int128)a * n + b) / c;
                if(!m) return {};

                Data t = calc(m - 1, c, c - b - 1, a);
```

```
                mint M = m % MOD;
                return {
                        N * M - t.f,
                        N * M * M - 2 * t.h - t.f,
                        M * s1 - (t.g + t.f) * inv2
                };
        }
// f: sum floor((a*i+b)/c)
// g: sum floor((a*i+b)/c)^2
// h: sum i*floor((a*i+b)/c)
}
```

7.3 exgcd

FILE math/number_theory/exgcd.cpp **LINES** 12

COST O(\log n)

AUTHOR yzj

```
// @brief
inline int exgcd(int a, int b, int &x, int &y) {
    if(!b) return x = 1, y = 0, a;
    int d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
inline int inv(int a, int mod) {
    int x, y;
    exgcd(a, mod, x, y);
    return (x % mod + mod) % mod;
}
```

7.4 min25

FILE math/number_theory/min25.cpp **LINES** 49

COST O(n^{3/4} / \log n)

AUTHOR yzj

```
// @brief
namespace min25 {
    int pri[N], cov[N], sq, m;
    int id1[N], id2[N], val[N], idx;
    mint f[N], g[N], h[N];
    map <pii, mint> dps;

    inline int id (int x) { return x <= sq ? id1[x] : id2[n / x]; }

    inline void init (int n) {
        sq = sqrt (n); m = idx = 0;

        rep (i, 2, sq) {
            if (!cov[i]) pri[++ m] = i, h[m] = h[m - 1] + i;
            for (int j = 1; j <= m && i * pri[j] <= sq; j ++) {
                cov[i * pri[j]] = 1;
                if (i % pri[j] == 0) break;
            }
        }

        for (int l = 1, r, v; l <= n; l = r + 1) {
            val[++ idx] = v = n / l; r = n / v;
            (v <= sq ? id1[v] : id2[n / v]) = idx;
            f[idx] = (mint) v * (v + 1) * inv2 - 1; g[idx] = v - 1;
        }

        rep (j, 1, m) for (int i = 1; i <= idx && pri[j] * pri[j] <= val[i]; i ++) {
            int k = id (val[i] / pri[j]);
            f[i] -= pri[j] * (f[k] - h[j - 1]);
            g[i] -= g[k] - j + 1;
        }
    }

    int TIME = 0;
    inline mint S (int n, int i) {
        if (n <= 1 || pri[i] >= n) return 0;

        int u = id (n);
        mint res = f[u] - g[u] - h[i] + i;
        if (i == 0) res += 2;
```

```
41
42          for (int j = i + 1; j <= m && pri[j] * pri[j] <= n; j ++) {
43              for (int c = 1, pw = pri[j]; pw <= n; pw *= pri[j], c ++)
44                  res += (pri[j] ^ c) * (S (n / pw, j) + (c > 1));
45          }
46
47          return res;
48      }
49  }
```

8 Misc

8.1 Default(Yzj)

FILE misc/default(yzj).cpp **LINES** 24

AUTHOR 喵喵喵幼儿园

```
1  #include<bits/stdc++.h>
2
3  #define int long long
4
5  #define vc vector
6  #define pb emplace_back
7  #define pii pair<int, int>
8  #define mkp make_pair
9  #define rep(i, a, b) for(int i = (a); i <= (b); ++i)
10 #define lep(i, a, b) for(int i = (a); i >= (b); --i)
11
12 using namespace std;
13
14 mt19937 gen(chrono::system_clock::now().time_since_epoch().count());
15
16 inline int read() {
17         int x = 0, w = 0; char ch = getchar(); while(!isdigit(ch)) w |= (ch == '-'), ch = getchar();
18         while(isdigit(ch)) x = x * 10 + (ch ^ 48), ch = getchar(); return w ? -x : x;
19 }
20
21 signed main() {
22
23         return 0;
24 }
```

8.2 快速模乘

FILE misc/fastmod.cpp **LINES** 16

USE 要求 1<=P<2^31 且 a<P；同一 z 多次使用时先 trans，再用 mul_t **COST** O(1)

AUTHOR ppip

```
1  using u32=unsigned;
2  using u64=unsigned long long;
3  using u128=unsigned __int128;
4  const u32 P=998244353;
5  inline u64 trans(u64 x) {
6      constexpr u64 A=-(u64)P/P+1;
7      constexpr u64 q=((u128(-(u64)P%P)<<64)+P-1)/P;
8      return x*A+u64((u128)x*q>>64)+1;
9  }
10 // t 必须是 trans(z) 的返回值；复用同一 z 时只需计算一次 t
11 inline u32 mul_t(u32 a,u64 t) {
12     return a*t*(u128)P>>64;
13 }
14 inline u32 mul(u32 a,u64 z) {
15     return mul_t(a,trans(z));
16 }
```

9 Poly

9.1 多项式求逆

FILE poly/finv.cpp **LINES** 97

USE ntt,多项式求逆 **COST** O(nlogn)

AUTHOR ysy

```
1  #include<iostream>
2  #include<cstdio>
3  #include<cstring>
```

```cpp
#define int long long
using namespace std;
int read(){
    int x=0,f=1;char ch=getchar();
    while(ch<'0'||ch>'9'){if(ch=='-')f=-1;ch=getchar();}
    while(ch>='0'&&ch<='9'){x=x*10+ch-'0';ch=getchar();}
    return x*f;
}
const int N=4000010,mod=998244353,g=3;
int ksm(int p,int k){
    int x=1;
    while(k){
        if(k&1)x=x*p%mod;
        p=p*p%mod;
        k>>=1;
    }
    return x;
}
int r[N];
void init(int lg){
    for(int i=0;i<(1<<lg);i++){
        r[i]=(r[i>>1]>>1)|((i&1)<<(lg-1));
    }
}
void ntt(int *a,int n,bool f=0){
    for(int i=0;i<n;i++){
        if(i<r[i])swap(a[i],a[r[i]]);
    }
    for(int ln=2;ln<=n;ln<<=1){
        int x=ksm(g,(mod-1)/ln),mid=(ln>>1);
        if(f)x=ksm(x,mod-2);
        for(int i=0;i<n;i+=ln){
            int y=1;
            for(int j=0;j<mid;j++){
                int l=a[i+j],r=y*a[i+j+mid]%mod;
                a[i+j]=(l+r)%mod,a[i+j+mid]=(l-r+mod)%mod;
                y=y*x%mod;
            }
        }
    }
    if(f){
        int iv=ksm(n,mod-2);
        for(int i=0;i<n;i++){
            a[i]=a[i]*iv%mod;
        }
    }
}
void mul(int *a,int *b,int n){
    static int c[N],d[N];
    memset(c,0,sizeof(c));
    memset(d,0,sizeof(d));
    int m=(n<<1);
    for(int i=0;i<n;i++){
        c[i]=a[i],d[i]=b[i];
    }
    ntt(c,m,0);ntt(d,m,0);
    for(int i=0;i<m;i++){
        a[i]=c[i]*d[i]%mod;
    }
    ntt(a,m,1);
}
void qinv(int n,int *g){
    int m=1,lg=0;
    static int f[N],h[N];
    memcpy(f,g,sizeof(f));
    memset(g,0,sizeof(g));
    g[0]=ksm(f[0],mod-2);
    while(m<=n){
        m<<=1,lg++;
        init(lg+1);
        memcpy(h,g,sizeof(h));
        memset(g,0,sizeof(g));
        for(int i=0;i<m;i++){
            g[i]=h[i]*2%mod;
        }
```

```
        mul(h,h,m);
        mul(h,f,m);
        for(int i=0;i<m;i++){
            g[i]=(g[i]-h[i]+mod)%mod;
        }
    }
}
int n,a[N],b[N];
signed main(){
    n=read()-1;
    for(int i=0;i<=n;i++){
        a[i]=read();
    }
    qinv(n,a);
    for(int i=0;i<=n;i++){
        cout<<a[i]<<" ";
    }
    return 0;
}
```

9.2 多项式乘法

FILE poly/ntt(ysy).cpp **LINES** 73

USE ntt,未经任何优化 **COST** O(nlogn)

AUTHOR ysy

```
#include<iostream>
#include<cstdio>
#define int long long
using namespace std;
int read(){
    int x=0,f=1;char ch=getchar();
    while(ch<'0'||ch>'9'){if(ch=='-')f=-1;ch=getchar();}
    while(ch>='0'&&ch<='9'){x=x*10+ch-'0';ch=getchar();}
    return x*f;
}
const int N=4000040,mod=998244353,g=3;
int n,m,a[N],b[N],r[N];
int ksm(int p,int k){
    int x=1;
    while(k){
        if(k&1)x=x*p%mod;
        p=p*p%mod;
        k>>=1;
    }
    return x;
}
void init(int lg){
    for(int i=1;i<(1<<lg);i++){
        r[i]=((r[i>>1]>>1)|((i&1)<<(lg-1)));
    }
}
void ntt(int *a,int lg,int fl=0){
    int n=(1<<lg);
    for(int i=0;i<n;i++){
        if(i<r[i])swap(a[i],a[r[i]]);
    }
    for(int ln=2;ln<=n;ln<<=1){
        int mid=(ln>>1),x=ksm(g,(mod-1)/ln);
        if(fl)x=ksm(x,mod-2);
        for(int i=0;i<n;i+=ln){
            int y=1;
            for(int j=0;j<mid;j++){
                int l=a[i+j],r=a[i+j+mid]*y%mod;
                a[i+j]=(l+r)%mod,a[i+j+mid]=(l-r+mod)%mod;
                y=y*x%mod;
            }
        }
    }
    if(fl){
        int iv=ksm(n,mod-2);
        for(int i=0;i<n;i++){
            a[i]=a[i]*iv%mod;
        }
    }
}
```

```
void mul(){
    int lg=__lg(n+m)+1;
    init(lg);
    ntt(a,lg);ntt(b,lg);
    for(int i=0;i<(1<<lg);i++){
        a[i]=a[i]*b[i]%mod;
    }
    ntt(a,lg,1);
}
signed main(){
    n=read(),m=read();
    for(int i=0;i<=n;i++){
        a[i]=read();
    }
    for(int i=0;i<=m;i++){
        b[i]=read();
    }
    mul();
    for(int i=0;i<=n+m;i++){
        cout<<a[i]<<" ";
    }
    return 0;
}
```

9.3 计数常用代码

FILE poly/ntt(yzj).cpp **LINES** 130

USE mint 类和 poly 类 **COST** O(n log n)

AUTHOR yzj

```
const int MOD = 998244353;

// CALC
inline void inc (int &x, int y) { x += y - MOD; x += (x >> 31) & MOD; }
struct mint {
        int x;
        mint (int ix = 0) { x = ix % MOD; if(x < 0) x += MOD; }
        mint operator += (mint a) { inc (x, a.x); return a; }
        mint operator -= (mint a) { inc (x, MOD - a.x); return a; }
        mint operator *= (mint a) { x = 1ll * x * a.x % MOD; return a; }
        mint friend operator + (mint a, mint b) { a += b; return a; }
        mint friend operator - (mint a, mint b) { a -= b; return a; }
        mint friend operator * (mint a, mint b) { a *= b; return a; }
} fc[N], ifc[N];
inline mint qpow (mint a, int b) { mint res = 1; for(; b; b >>= 1, a *= a) if(b & 1) res *= a; return res; }
inline mint qinv (mint a) { return qpow(a.x, MOD - 2); }
inline mint binom (int n, int m) { return n < m ? 0 : fc[n] * ifc[m] * ifc[n - m]; }
//

namespace Poly {
        const int MAXN = 1 << 22;
        int t, bit, rev[MAXN];

        struct poly : vc <mint> {
                poly (int n = 0) { resize (n); for (auto &u : (*this)) u = 0; }
                inline void show () {
                        for (auto u : (*this)) cerr << u.x << " ";
                        cerr << endl;
                }
        } ;

        inline void Resize (poly &a, int n) {
                while (a.size () < n) a.pb (0);
                while (a.size () > n) a.pop_back ();
        }
        inline void init (int n) {
                assert (n <= MAXN);
                t = 2, bit = 1; while (t < n) t <<= 1, ++ bit;
                rep (i, 0, t - 1) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (bit - 1));
        }
        inline void dft (poly &a) {
                rep (i, 0, t - 1) if (i < rev[i]) swap (a[i], a[rev[i]]);
                mint o = 1, w = qpow (3, (MOD - 1) >> 1), x, y;
                for (int m = 1; m < t; m <<= 1, w = qpow (3, (MOD - 1) / (m << 1)))
                        for (int i = 0; i < t; i += (m << 1), o = 1) for (int j = 0; j < m; j ++, o = o * w)
                                x = a[i + j], y = o * a[i + j + m], a[i + j] = x + y, a[i + j + m] = x - y;
```

```
        }
        inline void idft (poly &a) {
                dft (a); auto inv = qinv (t);
                reverse (a.begin () + 1, a.end ());
                for (auto &u : a) u *= inv;
        }
        poly operator * (poly a, poly b) {
                if (a.empty () || b.empty ()) return poly ();
                int h; init (h = a.size () + b.size () - 1);
                poly f (t); Resize (a, t); Resize (b, t);
                dft (a); dft (b);
                rep (i, 0, t - 1) f[i] = a[i] * b[i];
                idft (f); Resize (f, h);
                return f;
        }
        poly deriv (poly a) {
                int n = a.size ();
                poly b (n ? n - 1 : 0);
                rep (i, 1, n - 1) b[i - 1] = a[i] * i;
                return b;
        }
        poly integ (poly a) {
                int n = a.size ();
                static vc <mint> iv = {0, 1};
                for (int i = iv.size (); i <= n; i ++) iv.pb (mint (MOD - MOD / i) * iv[MOD % i]);
                poly b (n + 1);
                rep (i, 0, n - 1) b[i + 1] = a[i] * iv[i + 1];
                return b;
        }
        // Return the first n coefficients; omitted coefficients of a are zero.
        poly inv (poly a, int n) {
                if (!n) return poly ();
                assert (!a.empty () && a[0].x);
                Resize (a, n);
                poly b (1); b[0] = qinv (a[0]);
                for (int m = 1; m < n; m <<= 1) {
                        int k = min (m << 1, n);
                        poly c (k);
                        rep (i, 0, k - 1) c[i] = a[i];
                        c = c * b; Resize (c, k);
                        rep (i, 0, k - 1) c[i] = mint (0) - c[i];
                        c[0] += 2;
                        b = b * c; Resize (b, k);
                }
                return b;
        }
        poly ln (poly a, int n) {
                if (!n) return poly ();
                assert (!a.empty () && a[0].x == 1);
                if (n == 1) return poly (1);
                Resize (a, n);
                poly b = deriv (a) * inv (a, n - 1);
                Resize (b, n - 1);
                return integ (b);
        }
        poly exp (poly a, int n) {
                if (!n) return poly ();
                assert (a.empty () || !a[0].x);
                Resize (a, n);
                poly b (1); b[0] = 1;
                for (int m = 1; m < n; m <<= 1) {
                        int k = min (m << 1, n);
                        poly c = ln (b, k);
                        rep (i, 0, k - 1) c[i] = a[i] - c[i];
                        c[0] += 1;
                        b = b * c; Resize (b, k);
                }
                return b;
        }
}

int n, c[N], g[N];

inline Poly :: poly solve (int l, int r) {
        if (l == r) {
```

```
                Poly :: poly f (c[l] + 1);
                rep (i, 0, min (c[l],  g[l]))
                        if (i & 1) f[c[l] - i] = MOD - binom (c[l], i) * binom (g[l], i) * fc[i];
                        else f[c[l] - i] = binom (c[l], i) * binom (g[l], i) * fc[i];
                return f;
        }
        int mid = ((l + r) >> 1);
        return solve (l, mid) * solve (mid + 1, r);
}
```

9.4 DIF-DIT NTT & All op

FILE poly/poly.cpp **LINES** 189

USE 效率较高的 NTT **COST** 除『』外，均为 O(nlogn)

AUTHOR ppip

```
using LL=long long;
#define ensure(x) do { \
    if (!(x)) { \
        fprintf(stderr,"ensure failed: %s, line %d\n",#x,__LINE__); \
        exit(1); \
    } \
} while(0)
struct poly {
    static const int mod=998244353;
    vector<LL>a;
    static vector<LL>w;

    poly(){}
    poly(int n):a(n){}
    poly(vector<LL> v):a(move(v)){}
    poly(initializer_list<LL> v):a(v){}

    int size()const{return a.size();}
    LL& operator[](int i){return a[i];}
    const LL& operator[](int i)const{return a[i];}
    void resize(int n){a.resize(n);}

    static LL qp(LL a,LL n=mod-2){
        LL r=1;
        for(;n;n>>=1,a=a*a%mod)if(n&1)r=r*a%mod;
        return r;
    }

    static int up(int n){
        return n<=1?1:1<<(__lg(n-1)+1);
    }

    static void init(int n){
        ensure(n>0&&!(n&(n-1))&&n<=(1<<23));
        int m=n>>1,k=w.size();
        if(k>=m)return;
        w.resize(m);
        for(;k<m;k<<=1){
            LL r=qp(31,1<<(21-__lg(k)));
            for(int i=0;i<k;i++)w[k+i]=w[i]*r%mod;
        }
    }

    static void dif(vector<LL>&a){
        int n=a.size();
        init(n);
        for(int len=n>>1;len;len>>=1){
            int o=0;
            for(int l=0;l<n;l+=len<<1,++o){
                LL r=w[o];
                for(int i=l;i<l+len;i++){
                    LL x=a[i],y=r*a[i+len]%mod;
                    a[i]=x+y;
                    if(a[i]>=mod)a[i]-=mod;
                    a[i+len]=x-y;
                    if(a[i+len]<0)a[i+len]+=mod;
                }
            }
        }
    }
```

```
61
62      static void dit(vector<LL>&a){
63          int n=a.size();
64          init(n);
65          for(int len=1;len<n;len<<=1){
66              int o=0;
67              for(int l=0;l<n;l+=len<<1,++o){
68                  LL r=w[o];
69                  for(int i=l;i<l+len;i++){
70                      LL x=a[i],y=a[i+len];
71                      a[i]=x+y;
72                      if(a[i]>=mod)a[i]-=mod;
73                      a[i+len]=(x-y+mod)*r%mod;
74                  }
75              }
76          }
77      }
78
79      vector<LL> val(int n=0)const{
80          if(!n)n=up(size());
81          vector<LL>f(n);
82          copy_n(a.begin(),min(n,size()),f.begin());
83          dif(f);
84          return f;
85      }
86
87      static poly from(vector<LL>f){
88          if(f.empty())return {};
89          int n=f.size();
90          dit(f);
91          reverse(f.begin()+1,f.end());
92          LL iv=mod-(mod-1)/n;
93          for(LL&x:f)x=x*iv%mod;
94          return poly(move(f));
95      }
96
97      friend poly operator*(const poly&A,const poly&B){
98          if(!A.size()||!B.size())return {};
99          int s=A.size()+B.size()-1;
100         if(min(A.size(),B.size())<=32){
101             poly c(s);
102             for(int i=0;i<A.size();i++)for(int j=0;j<B.size();j++)
103                 c[i+j]=(c[i+j]+A[i]*B[j])%mod;
104             return c;
105         }
106         int n=up(s);
107         auto a=A.val(n);
108         if(&A==&B){
109             for(LL&x:a)x=x*x%mod;
110         }else{
111             auto b=B.val(n);
112             for(int i=0;i<n;i++)a[i]=a[i]*b[i]%mod;
113         }
114         poly c=from(move(a));
115         c.resize(s);
116         return c;
117     }
118
119     friend poly operator+(poly A,const poly&B){
120         if(A.size()<B.size()) A.resize(B.size());
121         for(int i=0;i<B.size();++i){
122             A[i]+=B[i];
123             if(A[i]>=mod) A[i]-=mod;
124         }
125         return A;
126     }
127     friend poly operator-(poly A,const poly&B){
128         if(A.size()<B.size()) A.resize(B.size());
129         for(int i=0;i<B.size();++i){
130             A[i]-=B[i];
131             if(A[i]<0) A[i]+=mod;
132         }
133         return A;
134     }
135
```

```
    poly& operator+=(const poly&b){
        return *this=*this+b;
    }
    poly& operator-=(const poly&b){
        return *this=*this-b;
    }
    poly& operator*=(const poly&b){
        return *this=*this*b;
    }

    friend poly inv(const poly &f,int n=-1) {
        ensure(f.size() && f[0]);
        if (n==-1) n=(f.size()-1);
        poly F{f[0]},G{qp(F[0])};
        for (int i{2};(i>>1)<=n;i<<=1) {
            // FG=1+E
            // G1=G(1-E)=G(1-(FG-1))=G(2-FG)
            F.resize(i);
            for (int j{i/2};j<i&&j<=n;++j) F[j]=f[j];
            G=G*(poly{2}-F*G);
            G.resize(i);
        }
        G.resize(n+1);
        return G;
    }
    friend poly dev(const poly &f) {
        if (f.size()<=1) return poly{0};
        poly g(f.size()-1);
        for (int i=1;i<f.size();++i)
            g[i-1]=f[i]*i%mod;
        return g;
    }
    friend poly inte(const poly &f) {
        poly g(f.size()+1);
        g[0]=0;
        vector<LL> iv(f.size()+1);
        if (f.size()) iv[1]=1;
        for (int i=2;i<=f.size();++i)
            iv[i]=mod-mod/i*iv[mod%i]%mod;
        for (int i=0;i<f.size();++i)
            g[i+1]=f[i]*iv[i+1]%mod;
        return g;
    }
    friend poly ln(const poly &f,int n=-1) {
        ensure(f[0]==1);
        // G'=(lnF)'=F'/F
        // G=int(F'*F^-1)
        if (n==-1) n=f.size();
        auto g{inte(dev(f)*inv(f))};
        g.resize(n);
        return g;
    }
};
vector<LL> poly::w{1};
```

10 String

10.1 广义 SAM

FILE string/exsam.cpp **LINES** 62

USE 建立多串 SAM **COST** O(n*C)

AUTHOR ysy

```
#include<iostream>
#include<cstdio>
#include<cstring>
#define ll long long
using namespace std;
int read(){
    int x;
    scanf("%d",&x);
    return x;
}
const int N=2000020;
struct SAM{
```

```
    int fa[N],ch[N][26],ln[N],ct=1,la=1;
    void ins(int c){
        int p=la;
        if(ch[p][c]){
            int q=ch[p][c];
            if(ln[q]==ln[p]+1)la=q;
            else{
                int nq=++ct;ln[nq]=ln[p]+1,la=nq;
                memcpy(ch[nq],ch[q],sizeof(ch[q]));
                fa[nq]=fa[q];fa[q]=nq;
                for(;p&&ch[p][c]==q;p=fa[p])ch[p][c]=nq;
            }
            return;
        }
        int np=++ct;ln[np]=ln[p]+1,la=np;
        for(;p&&!ch[p][c];p=fa[p])ch[p][c]=np;
        if(!p)fa[np]=1;
        else{
            int q=ch[p][c];
            if(ln[q]==ln[p]+1)fa[np]=q;
            else{
                int nq=++ct;ln[nq]=ln[p]+1;
                memcpy(ch[nq],ch[q],sizeof(ch[q]));
                fa[nq]=fa[q];fa[q]=fa[np]=nq;
                for(;p&&ch[p][c]==q;p=fa[p])ch[p][c]=nq;
            }
        }
    }
    ll calc(){
        ll ans=0;
        for(int i=2;i<=ct;i++){
            ans+=ln[i]-ln[fa[i]];
        }
        return ans;
    }
}sa;
int n,m;
char c[N];
int main(){
    n=read();
    for(int i=1;i<=n;i++){
        scanf("%s",c+1);
        m=strlen(c+1),sa.la=1;
        for(int j=1;j<=m;j++){
            sa.ins(c[j]-'a');
        }
    }
    cout<<sa.calc()<<"\n"<<sa.ct;
    return 0;
}
```

10.2 manacher

FILE string/manacher.cpp **LINES** 13

USE 线性求回文子串数量 **COST** O (n)

AUTHOR yzj

```
namespace Manacher {
    int r[N], L, R, res;
    inline int manacher(char *s, int n) {
        res = L = R = 0;
        rep(i, 1, n) {
            r[i] = i <= R ? min(r[R + L - i], R - i) : 0;
            while(i + r[i] + 1 <= n && i - r[i] - 1 > 0 && s[i + r[i] + 1] == s[i - r[i] - 1]) ++r[i];
            if(i + r[i] > R) L = i - r[i], R = i + r[i];
            res = max(res, r[i] / 2 * 2 + (i % 2 == 0));
        }
        return res;
    }
}
```

10.3 PAM

FILE string/pam.cpp **LINES** 27

USE 回文树 **COST** O(n)

AUTHOR yzj

```cpp
namespace PAM {
    int ch[N][26], fail[N], len[N], cnt[N], last, tot;
    inline int getfail(int p, int x) {
        while(str[x - len[p] - 1] != str[x]) p = fail[p];
        return p;
    }
    inline void insert(int x) {
        int c = str[x] - 'a', p = getfail(last, x);
        if(!ch[p][c]) {
            len[++tot] = len[p] + 2;
            if(len[tot] == 1) fail[tot] = 0;
            else {
                int q = getfail(fail[p], x);
                fail[tot] = ch[q][c];
            }
            ch[p][c] = tot;
        }
        last = ch[p][c]; cnt[last]++;
    }
    inline void build() {
        len[0] = 0; fail[0] = 1;
        len[1] = -1; fail[1] = 1;
        last = 0; tot = 1;
        for(int i = 1; i <= n; i++) insert(i);
        for(int i = tot; i > 1; i--) cnt[fail[i]] += cnt[i];
    }
}
```

10.4 SA

FILE string/sa(oscar).cpp **LINES** 41

USE 后缀排序并求出 height 数组 **COST** O(n \log n)

AUTHOR yzj

```cpp
namespace SA {
    int sa[N], rk[N], tmp[N], id[N], buc[N], height[N], idx;
    inline void suffix_sort() {
        for(int i = 0; i < 256; i++) buc[i] = 0;
        for(int i = 1; i <= n; i++) buc[(unsigned char)str[i]]++;
        for(int i = 1; i < 256; i++) buc[i] += buc[i - 1];
        for(int i = n; i; i--) sa[buc[(unsigned char)str[i]]--] = i;
        rk[sa[1]] = idx = 1;
        for(int i = 2; i <= n; i++)
            rk[sa[i]] = str[sa[i]] == str[sa[i - 1]] ? idx : ++idx;

        for(int w = 1; idx < n; w <<= 1) {
            int p = 0;
            for(int i = n - w + 1; i <= n; i++) id[++p] = i;
            for(int i = 1; i <= n; i++) if(sa[i] > w) id[++p] = sa[i] - w;

            for(int i = 1; i <= idx; i++) buc[i] = 0;
            for(int i = 1; i <= n; i++) buc[rk[id[i]]]++;
            for(int i = 1; i <= idx; i++) buc[i] += buc[i - 1];
            for(int i = n; i; i--) sa[buc[rk[id[i]]]--] = id[i];

            for(int i = 1; i <= n; i++) tmp[i] = rk[i];
            rk[sa[1]] = idx = 1;
            for(int i = 2; i <= n; i++) {
                int x = sa[i], y = sa[i - 1];
                rk[x] = tmp[x] == tmp[y] &&
                    (x + w > n ? 0 : tmp[x + w]) == (y + w > n ? 0 : tmp[y + w])
                    ? idx : ++idx;
            }
        }

        for(int i = 1, k = 0; i <= n; i++) {
            if(rk[i] == 1) { k = 0; continue; }
            if(k) k--;
            int j = sa[rk[i] - 1];
            while(i + k <= n && j + k <= n && str[i + k] == str[j + k]) k++;
            height[rk[i]] = k;
        }
        height[1] = 0;
    }
}
```

10.5 后缀数组

FILE string/sa.cpp **LINES** 34

USE 倍增算法求 sa，rk，height, 字符必须不超过 '~' **COST** O(n log n)

AUTHOR ppip

```
char s[N+5];
int id[N*2+5],sa[N+5],buc[N+5],h[N+5],rk[N+5];
int main() {
    // read s
    int n(strlen(s+1));
    for (int i{1};i<=n;++i) ++buc[s[i]];
    partial_sum(buc+1,buc+1+'~',buc+1);
    for (int i{n};i>=1;--i) sa[buc[s[i]]--]=i;
    int m{0};
    for (int i{1};i<=n;++i) rk[sa[i]]=m+=(s[sa[i]]!=s[sa[i-1]]);
    fill_n(buc+1,'~',0);
    for (int w{1};m<n;w<<=1) {
        int cnt{0};
        for (int i{0};i<w;++i) id[++cnt]=n-i;
        for (int i{1};i<=n;++i) {
            if (sa[i]>w) id[++cnt]=sa[i]-w;
            ++buc[rk[i]];
        }
        partial_sum(buc+1,buc+1+m,buc+1);
        for (int i{n};i>=1;--i) sa[buc[rk[id[i]]]--]=id[i];
        fill_n(buc+1,m,0);
        copy_n(rk+1,n,id+1);
        m=0;
        for (int i{1};i<=n;++i)
            rk[sa[i]]=m+=(id[sa[i]]!=id[sa[i-1]]||id[sa[i]+w]!=id[sa[i-1]+w]);
    }
    // for (int i{1};i<=n;++i) cout<<sa[i]<<" ";
    for (int i{1},k{0};i<=n;++i) {
        k-=!!k;
        while (s[i+k]==s[sa[rk[i]-1]+k]) ++k;
        h[rk[i]]=k;
    }
    return 0;
}
```

10.6 SAM

FILE string/sam.cpp **LINES** 36

USE Magic automaton **COST** O(n)

AUTHOR ppip

```
constexpr int N(1e6),S{26};
struct SAM {
    int cnt,cur,n;
    struct node {
        int len,link;
        array<int,S> ch;
    } t[N*2+5];
    SAM() {cur=cnt=1;fill(t[0].ch.begin(),t[0].ch.end(),1);t[0].len=-1;}
    void extend(int c) {
        ++n;
        int x{cur},p{++cnt};
        t[p].len=n;
        while (!t[x].ch[c]) t[x].ch[c]=p,x=t[x].link;
        int q{t[x].ch[c]};
        t[p].link=q;
        if (t[x].len+1!=t[q].len) {
            int nq{++cnt};
            t[nq].ch=t[q].ch;
            t[nq].link=t[q].link;
            t[nq].len=t[x].len+1;

            t[p].link=t[q].link=nq;
            while (t[x].ch[c]==q) t[x].ch[c]=nq,x=t[x].link;
        }
        cur=p;
    }
    void print() {
        return;
        for (int i{1};i<=cnt;++i)
            for (int j{0};j<S;++j)
```

```
                if (t[i].ch[j])
                    cerr<<i<<" "<<t[i].ch[j]<<" "<<char(j+'a')<<endl;
        for (int i{cur};i;i=t[i].link) cerr<<i<<" ";
        cerr<<endl<<endl;
    }
} T;
```

10.7 后缀树

FILE string/sft.cpp **LINES** 39

USE Ukkonen 在线构造隐式后缀树 **COST** build O(n)

AUTHOR ppip

```
constexpr int N(1e6),inf(1e7),RNG{26+26+10};
struct Suft {
    int ch[N*2+5][RNG];
    int st[N*2+5],len[N*2+5],link[N*2+5];
    char s[N+5];
    int now{1},rem{0},n{0},tot{1};
    Suft() {len[0]=inf;}
    int new_node(int l,int le) {
        ++tot;st[tot]=l;len[tot]=le;
        return tot;
    }
    void extend(int x) {
        s[++n]=x;++rem;
        for (int lst{1};rem;) {
            while (rem>len[ch[now][s[n-rem+1]]])
                rem-=len[now=ch[now][s[n-rem+1]]];
            int &v{ch[now][s[n-rem+1]]},c{s[st[v]+rem-1]};
            if (!v||x==c) {
                link[lst]=now;lst=now;
                if (!v) v=new_node(n,inf);
                else break;
            } else {
                int u{new_node(st[v],rem-1)};
                ch[u][c]=v;ch[u][x]=new_node(n,inf);
                st[v]+=rem-1;len[v]-=rem-1;
                link[lst]=v=u;lst=u;
            }
            if (now==1) --rem;
            else now=link[now];
        }
    }
    void search(int u,int dep=0) {
        if (st[u]+len[u]-1>=n&&st[u]-dep!=n) cout<<st[u]-dep<<" ";
        else dep+=len[u];
        for (int i{0};i<RNG;++i)
            if (ch[u][i]) search(ch[u][i],dep);
    }

} T;
```

警钟长鸣

读题与建模

! 多模拟样例

思考误区

! 发现新性质后，回头检查旧做法能否简化或是否已经不成立

! DP 先想清楚有哪些可靠的状态和存储方式，再设计转移

实现与边界

! 任意范围修改、查询时，考虑复杂度会不会退化

提交之前

! 检查多测清空

! 检查 long long 和取模

! TLE 时重新检查实际复杂度，尤其注意分治树等结构是否退化